

Embedded Roadmap

(@stuck_at_ritesh)

1. C Programming Fundamentals

This is the MOST important screening area.

Core Topics

A. Pointers

You must deeply understand:

- Pointer arithmetic
- Pointer to pointer
- Function pointers
- Array vs pointer difference
- Pointer aliasing
- Memory access patterns

B. Data Structures

You should implement:

- Arrays
- Linked Lists
- Stacks
- Queues
- Hash tables
- Binary Search Trees
- Graph basics

C. Algorithms

Must know:

- Sorting
- Searching
- Recursion
- Time complexity
- Space complexity

What companies Actually Tests

- Can you manipulate memory safely?
- Can you reason about addresses?
- Can you debug mentally?
- Can you optimize runtime?
- Can you write bug-free logic?=

Recommended Resources

Books

- The C Programming Language - Kernighan and Ritchie
- Expert C Programming
- Pointers on C

Practice

- LeetCode (easy-medium)
- HackerRank C section
- Write every DS manually in C

2. Embedded C

This is where embedded interviews become different from software interviews.

Must-Know Topics

A. Memory Layout of C Program

Understand:

- Stack
- Heap
- Data segment
- BSS segment
- Text/code segment

You must explain:

- Where global/static variables go
- Where string literals go
- Why stack overflow happens

B. volatile Keyword

Critical embedded topic.

You must know:

- Hardware registers
- ISR variables
- Multi-thread/shared variables

Typical interview:

“Why volatile for hardware registers?”

C. static Keyword

Need clarity on:

- File scope
- Function scope
- Persistent lifetime

D. Struct Padding & Alignment

Must know:

- Why padding exists
- Memory alignment
- Packed structs
- Performance tradeoffs

E. Bit Manipulation

Very high priority.

Must know:

- Set bit
- Clear bit
- Toggle bit
- Check bit
- Bit masks

Example: `reg |= (1 << 5);`

F. Compilation Stages

Need clarity on:

1. Preprocessing
2. Compilation
3. Assembly
4. Linking
5. Loading

Recommended Resources

- Embedded Systems: An Integrated Approach - Layla B Das
- Embedded C tutorials
- Bare-metal MCU programming projects

3. Operating Systems

Embedded interviews focus on OS concepts from practical angle.

Core Topics

A. Scheduling Algorithms

Need clarity on:

- FCFS
- Round Robin
- Priority Scheduling
- Preemptive vs Non-preemptive

B. Processes vs Threads

Critical topic.

Must explain:

- Context switching
- Stack separation
- Shared memory

C. Synchronization

Very important.

Must know:

- Mutex
- Semaphore
- Spinlock
- Deadlock
- Race condition

D. Memory

Need understanding of:

- Virtual memory

- Paging
- Fragmentation
- Stack vs heap

RTOS Concepts

Strong advantage if you know:

- Task scheduling
- ISR handling
- Priority inversion
- Deterministic timing

Recommended Resources

- Operating Systems - Andrew S Tanenbaum
- FreeRTOS documentation
- RTOS YouTube lectures - [Introduction To Embedded System Explained in Hindi | Embedded and ...](#)

4. Computer Architecture

This section differentiates strong firmware engineers.

Must-Know Topics

A. CPU Pipeline

Need understanding of:

- Fetch
- Decode
- Execute
- Pipeline hazards

B. Cache Memory

Very important.

Need clarity on:

- Cache hit/miss
- Locality
- Associativity
- Write-through vs write-back

C. DMA

TI LOVES DMA questions.

Need to know:

- Why DMA exists
- CPU offloading
- Memory-peripheral transfer

D. Memory Organization

Must know:

- ROM
- RAM

- Flash
- EEPROM

E. Interrupts

Critical embedded topic.

Need clarity on:

- ISR
- Interrupt latency
- Priorities
- Nesting

Recommended Resources

- Computer Organization and Architecture - William Stallings

5. Microcontrollers & Interfaces

This is the practical embedded engineering section.

High-Priority Topics

A. 8051 Basics

Need:

- Architecture
- Registers
- Memory organization
- Timers
- Interrupts

B. GPIO

Must know:

- Input/output modes
- Pull-up/pull-down
- Debouncing

C. Timers

Need clarity on:

- Timer overflow
- PWM generation
- Delays

D. ADC & DAC

Must understand:

- Sampling
- Resolution

- Quantization

E. Communication Protocols

SPI

- Master/slave
- Clocking
- Full duplex

UART

- Baud rate
- Framing

I2C

- Addressing
- ACK/NACK

Suggested Project Roadmap

Take inspiration from these projects and make something unique.

Beginner

- LED blinking
- Button interfacing
- Timer interrupt
- UART hello world

Intermediate

- SPI sensor interface
- ADC reading
- PWM motor control
- EEPROM communication

Advanced

- RTOS scheduler
- Bootloader
- DMA-based UART
- Low-power firmware
- Driver development

Sample Problem 1.1

For a deeply embedded processor such that it:

- do not has any cache
- Has three memories such that in terms of speed they are:
L1 RAM > L2 RAM > FLASH
- On startup it zero-initialize its .bss section (a place where all un-initialize variables are placed)

identify which implementations would be the slowest and fastest?

Program Section Name	Target Memory
.data	L2 RAM
.bss	L1 RAM
.stack	L1 RAM
.heap	L2 RAM
.code	FLASH
.rodata	FLASH

TI Information – Selective Disclosure

```
Implementation 1
int global_sum = 0;
compute_sum(range)
{
    for value in range {
        global_sum = global_sum + value
    }
}
```

```
Implementation 2
int global_sum = 0;
compute_sum(range)
{
    int local_sum = 0;
    for value in range {
        local_sum = local_sum + value
    }
    global_sum = local_sum;
}
```

```
Implementation 3
int global_sum = 0;
compute_sum(range)
{
    static int local_sum = 0;
    for value in range {
        local_sum = local_sum + value
    }
    global_sum = local_sum;
}
```

6

Solution 1.1

Here, we have to remember the following

Program Section Name	Contains
.data	All initialized global sections and initialized local static
.bss	All uninitialized global sections and uninitialized local static
.stack	Call stack
.heap	Dynamic allocation
.text	Code
.rodata	Read only data

Therefore, implementation 3 would be slowest, since, this implementation access variables stored in .data section a greater number of times.

Implementation 2 is the fastest because, even without any compiler optimization, local_sum would be stored in .stack which is present in L1RAM.

Sample Problem 1.2

Identify the error in the implementation:

```
int length;
void foo(int sg)
{
    int reverse_sg;
    while(sg>0)
    {
        length = length + 1;
        reverse_sg = reverse_sg * 10 + (sg%10);
        sg = sg / 10;
    }
}
```

Solution 1.2

- Here the problem is with file `reverse_sg = reverse_sg * 10 + (sg%10)`; because `reverse_sg` is a local variable which could be stored in either stack or in CPU register bank and in either case, it will have garbage value, however, `length` variable will never be a problem because program on startup is zero-initializing the bss section.

Sample Problem 2

Consider a high performance application which is running on CortexA72. It has many different functions

1. it has a function that loops over every element of a linear data-structure starting from a beginning to end. To reduce the run time of this function, which linear data structure should it loop onto?
2. It has a function that does computation of a 2d matrix. Should that matrix be computed in column-major fashion or row-major fashion in order to reduce run-time? Why?
3. In a linked-list, how would you detect a loop?

Solution 2

1. Array because it has good special locality.
2. Row-major because it has good special locality.
3. Floyd's cycle-finding algorithm is used for this.

Sample Problem 3

An Industrial memory access controller access a memory bank which is 16K-bytes in size and is organized as 16K × 8. During testing, it was found that

1. At normal temperature, it works as per its design i.e. it reads and writes 4 bytes at a time.
2. at elevated temperature of 105 deg C, it writes 4 bytes correctly but reads 4 bytes with Least-Significant-Byte as garbage.

Write functions in C that read 4 bytes of data at a time at elevated temperature.

Solution 3

- This is to test bit manipulation.

```
// function to read bytes at normal temperature.
uint32_t read4bytes(uint32_t address)
{
    return MemoryController_HAL_get4bytes(address);
}

// function to read bytes at elevated temperature.
uint32_t read4bytes_ex(uint32_t address)
{
    uint32_t hi = read4bytes(address);
    uint32_t lo = read4bytes(address + 1);
    uint32_t res = (hi & ((uint32_t)(~0xff))) | ((lo >> 8) & ((uint32_t)(0xff)));
    return res;
}
```

100 Embedded/Firmware Interview Questions

C Programming

1. Difference between array and pointer?
2. Explain pointer arithmetic.
3. What is dangling pointer?
4. What is wild pointer?
5. Difference between malloc and calloc?
6. What happens during free()?
7. Difference between stack and heap?
8. Explain function pointers.
9. What are static variables?
10. Explain recursion internally.
11. Why are arrays passed as pointers?
12. What is segmentation fault?
13. Explain memory leak.
14. What is undefined behavior?
15. Explain const correctness.
16. Difference between char *p and char p[]?
17. What happens in buffer overflow?
18. Difference between pass-by-value and pass-by-reference?
19. Explain typedef.
20. What is volatile?

Embedded C

21. Why use volatile for hardware registers?
22. Explain memory-mapped I/O.
23. What is ISR?
24. Why should ISRs be short?
25. Explain reentrancy.
26. What is race condition?
27. What is atomic operation?
28. Explain struct padding.
29. Why alignment matters?
30. What is packed structure?
31. Difference between static and extern?
32. Explain linker role.
33. What happens during preprocessing?
34. Difference between macro and inline?
35. What is conditional compilation?

- 36. Why are macros dangerous?
- 37. Explain bit masking.
- 38. Write code to set 5th bit.
- 39. Write code to clear 3rd bit.
- 40. Explain endianness.

Operating Systems

- 41. Difference between process and thread?
- 42. What is context switching?
- 43. Explain scheduling algorithms.
- 44. What is deadlock?
- 45. Conditions for deadlock?
- 46. Explain semaphore.
- 47. Difference between mutex and semaphore?
- 48. What is starvation?
- 49. Explain priority inversion.
- 50. What is paging?
- 51. What is virtual memory?
- 52. Difference between user mode and kernel mode?
- 53. What is interrupt latency?
- 54. Explain IPC mechanisms.
- 55. What is shared memory?
- 56. Explain cache coherence.
- 57. Difference between polling and interrupts?
- 58. What is real-time OS?
- 59. Difference between hard and soft real-time systems?
- 60. Why deterministic timing matters?

Computer Architecture

- 61. Explain instruction pipeline.
- 62. What are pipeline hazards?
- 63. What causes cache misses?
- 64. Difference between SRAM and DRAM?
- 65. Explain DMA.
- 66. Why DMA improves performance?
- 67. Explain memory hierarchy.
- 68. What is cache locality?
- 69. Difference between Harvard and Von Neumann architecture?

70. Explain branch prediction.
71. What is MMU?
72. Difference between physical and virtual address?
73. Explain TLB.
74. What is bus arbitration?
75. Explain interrupt vector table.
76. What is throughput vs latency?
77. Explain clock skew.
78. Why are caches smaller but faster?
79. What limits CPU frequency scaling?
80. Explain speculative execution.

Microcontrollers

81. Explain GPIO internally.
82. What are pull-up resistors?
83. Explain timer interrupts.
84. Difference between polling and interrupt-driven systems?
85. Explain PWM generation.
86. What is watchdog timer?
87. Explain ADC quantization.
88. What determines ADC resolution?
89. Explain DAC operation.
90. Difference between SPI and I2C?
91. Explain UART framing.
92. Why SPI is faster than I2C?
93. What is clock stretching in I2C?
94. Explain bootloader.
95. What happens during MCU reset?
96. Explain brownout condition.
97. What is debounce?
98. Explain low-power modes.
99. What happens during interrupt servicing?
100. Design firmware for a sensor node with low power and periodic communication.

Most Important Embedded Interview Skill

The strongest embedded engineers:

- Think at hardware level
- Understand timing
- Reason about memory
- Understand concurrency
- Write safe low-level code
- Debug systematically

Interviewers LOVE candidates who explain:

“What physically happens inside hardware when software executes.”

That mindset is what separates strong firmware engineers from regular coders.

The syllabus mentioned here, majorly overlaps with what TI had provided us during the hiring process.

Best of Luck Guys

Stay Hungry

ALWAYS